Proposal: Unified Token Architecture for Large-Scale CVR Prediction

QQ Algorithm Challenge — TAAC & KDD 2026

algo.qq.com

Ben Tsai Yu Xia David Ewing

April 2026

Proposal v0.01 — For Team Review

Abstract

I am writing this proposal to outline how we might approach the QQ Algorithm Challenge. These are just a set of ideas I am suggesting, rather than starting with a blank sheet. I invite everyone to review, challenge, and/or agree on the direction before we begin implementation — but whatever we decide, let us keep it in a current document, even if it changes, until such time as we agree we understand it well enough that we no longer need to keep a living document updated.

The contest goal appears to be providing a unification of Sequence Modelling and Feature Interaction for Large-scale Recommendation. It appears to be a reaction to the fact that two concepts have been diverging over time, and that there may be some benefit in putting something together that unifies them:

- **Feature-interaction models** — high-dimensional multi-field categorical and contextual features (FM[1], DeepFM[2], DCN[3], DIN[4]).
- **Sequential models** — temporal dynamics of user behaviour via Transformer-style ranking models (SASRec[5], BERT4Rec[6]).

I propose a unified token representation that places sequential behaviour events and non-sequential multi-field features into a single homogeneous token stream, processed by a shared stackable Transformer backbone. I believe this approach directly addresses both the leaderboard metric (AUC of ROC on CVR prediction) and the two innovation awards (Unified Block Innovation; Scaling Law Innovation).

Keywords: unified architecture, tokenisation, CVR prediction, transformers, feature interaction, scaling laws, recommender systems

[1]FM: Factorisation Machine. Models pairwise feature interactions via inner products of learned embedding vectors. Rendle, 2010.

[2]DeepFM: Deep Factorisation Machine. Combines an FM component with a deep neural network to capture both low-order and high-order feature interactions. Guo et al., 2017.

[3]DCN: Deep & Cross Network. Uses explicit cross layers to learn bounded-degree feature interactions efficiently alongside a deep network. Wang et al., 2017.

[4]DIN: Deep Interest Network. Uses an attention mechanism to adaptively weight historical user behaviours relative to the target item. Zhou et al., 2018.

[5]SASRec: Self-Attentive Sequential Recommendation. Applies a unidirectional Transformer to model sequential user behaviour for next-item prediction. Kang & McAuley, 2018.

[6]BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformers. Adapts the BERT masked-language-model objective to learn bidirectional user behaviour representations. Sun et al., 2019.

1 Problem Framing

As I see it from a bit of research and from the goals of this contest, industrial recommender systems have evolved into two independent paradigms:

- **Feature-interaction models** (FM, DeepFM, DCN, DIN) — focus on high-dimensional multi-field categorical and contextual features; optimise cross-feature interactions.
- **Sequential models** (GRU4Rec, SASRec, BERT4Rec, Transformer-based rankers) — focus on temporal dynamics of user behaviour; encode ordered item sequences.

Both paradigms have achieved success, yet they have been optimised separately. In my view, this creates three structural problems that I believe we should address:

1. **Shallow cross-paradigm interaction.** Concatenation of a sequence embedding with feature embeddings discards the cross-attention signal between the two token types.
2. **Inconsistent objectives.** Joint training of two architecturally distinct sub-networks introduces competing gradient signals and higher engineering complexity.
3. **Limited scalability.** Separate encoders do not scale uniformly; hardware efficiency is compromised by heterogeneous compute paths.

2 Proposed Approach: UniToken Architecture

2.1 Core Idea

The central idea is simple: stop treating user history and user attributes as two separate inputs that get glued together at the end. Instead, represent everything — past interactions, profile fields, and the candidate item — as items in a single list, and let one shared network read the whole list at once.

Concretely, I propose assembling a single sequence of tokens $\mathcal{T}$:

- the user's recent interactions, ordered by time: $[t_1, t_2, \ldots, t_L]$,
- the user's static feature fields (age group, region, device, etc.): $[f_1, f_2, \ldots, f_K]$,
- the candidate item being scored: $[q]$.

All three groups are placed end-to-end into one list and fed into a single Transformer. The $\|$ symbol below just means "join these lists one after the other":

$$\mathcal{T} = \underbrace{[t_1, t_2, \ldots, t_L]}_{\text{behaviour sequence}} \;\|\; \underbrace{[f_1, f_2, \ldots, f_K]}_{\text{non-sequential feature fields}} \;\|\; \underbrace{[q]}_{\text{target item / query}}$$

Every token in the list has the same size (embedding dimension d), so the Transformer can treat them all uniformly.

Is this a time series approach? No — and the distinction is important for the team. A time series model (e.g. ARIMA or an LSTM forecaster) asks "how will this measurement change over time?" and tries to predict the next value of the same signal. What we are doing is different: the ordered history $[t_1, \ldots, t_L]$ provides context about what the user has found relevant in the past, but the question we are answering is "will this user convert on this specific item q?" — a binary yes/no classification, not a forecast. The Transformer does not model whether behaviour is trending up or down; it models which parts of the history are *relevant* to the candidate item. This distinction affects how we set up positional embeddings and the training loss.

2.2 Unified Token Embedding

Before placing everything into one list, each token needs to be converted into a fixed-length vector of numbers — this is called an *embedding*. The key point is that every token, regardless of whether it is a past interaction, a feature field, or the candidate item, is converted using the *same* process. Nothing is special-cased.

Each token's vector is the sum of three parts:

1. **What it is** — a learned vector for the token's identity (e.g. "item 42", "region: NZ").
2. **Where it sits** — a learned vector encoding its position: time order for behaviour tokens, field index for feature tokens.
3. **What type it is** — a learned vector marking whether this token is from the behaviour sequence, the feature fields, or is the query item.

Summing these three parts gives the final vector $\mathbf{e}_i$ for token i:

$$\mathbf{e}_i = \text{Embed}(v_i) + \mathbf{p}_i + \mathbf{s}_i$$

All three parts have the same size d, so adding them is straightforward. The type marker $\mathbf{s}_i$ is what lets the Transformer tell the three groups apart, even though they share the same architecture.

2.3 Shared Stackable Backbone

Once every token has been converted to a vector, the full list is passed through a stack of N identical Transformer blocks. Each block does two things:

1. **Attention** — every token looks at every other token and decides how much to "borrow" information from it. This is how a past interaction can influence how the model reads a feature field, and vice versa.
2. **Feed-forward update** — each token's vector is updated independently using a small two-layer network.

Critically, *every block treats all token types identically.* There is no separate block for behaviour tokens and another for feature tokens — just N identical blocks stacked on top of each other, each attending across the entire list:

$$\mathbf{H}^{(n)} = \text{TransformerBlock}\left(\mathbf{H}^{(n-1)}\right), \qquad n = 1, \ldots, N$$

N is a design choice we can tune — more blocks means more capacity but also more compute. The scaling experiments in Section 3 are designed to understand this trade-off.

Implementation note — non-reentrant blocks. Each of the N blocks will be a *separate* instance with its own independently learned weights, not a single block whose weights are reused across passes. This is the standard Transformer design (as in BERT and GPT) and the reason it matters here is twofold. First, separate weights allow each layer to learn qualitatively different transformations of the token representations — early layers may resolve local token relationships, later layers higher-order ones. Second, and more practically for this contest, the depth-scaling ablations ($N = 1, 2, 4, 8$) only produce a meaningful parameter-count and compute curve if each depth level genuinely adds new parameters. If a single block's weights were shared across all N passes (a design sometimes called a Universal Transformer), then $N = 8$ and $N = 1$ would have identical parameter counts, and any scaling law

we reported would be an artefact of inference compute rather than model capacity — which is not the claim we want to make for the Scaling Law Innovation Award.

2.4 Prediction Head

After the N Transformer blocks have processed the full token list, we only need to look at one token's output vector: the query token q (the candidate item). By this point, that vector has been enriched by attending to the user's entire history and all feature fields.

We pass it through a single linear layer followed by a sigmoid function, which squashes the result to a number between 0 and 1 — our predicted conversion probability:

$$\hat{y} = \sigma\left(\mathbf{w}^\top \mathbf{h}_q^{(N)} + b\right).$$

In plain terms: if $\hat{y}$ is close to 1, the model thinks the user will convert; close to 0, it thinks they will not. We train by comparing $\hat{y}$ to the actual outcome (1 = converted, 0 = did not) using binary cross-entropy loss.

2.5 Where This Sits Relative to Prior Work

I cannot yet claim with confidence that this architecture is the first of its kind. What I can say is that ChatGPT has shown me that:

- DIN, SIM, TWIN — use the target item to gate the behaviour sequence via attention, but still process feature fields through a separate encoder.
- ETA, HSTU — introduce structural changes to the sequence encoder (e.g. efficient attention, hierarchical units) but, as far as I can determine, do not place feature fields into the same token stream as the sequence.

The contest description also references papers [1–3] as work that "begins to bridge the two branches". I have not yet read these in detail. **Before we can make any novelty claim, we should read those papers and check whether any of them already proposes a fully unified tokenisation.**

Placeholder: Literature Comparison Table

This subsection is a placeholder. Once the team has reviewed the contest references [1–3] and any other closely related work, we should replace this with a short comparison table along the lines of:

Approach	Unified token stream?	Shared backbone?	Notes
DIN / SIM / TWIN	No	No	Separate seq. encoder
ETA / HSTU	Partial	No	Seq. only
[1]–[3]	TBD	TBD	To be reviewed
UniToken (ours)	**Yes**	**Yes**	*claim to verify*

If any of [1–3] already proposes the same scheme, the novelty claim in this proposal will need to be revised accordingly.

3 Evaluation Plan

3.1 Primary Metric

AUC of ROC on the competition CVR prediction task.

3.2 Ablations

Table 1 describes the planned ablation study to isolate the contribution of each design choice.

Table 1: Planned ablations

Variant	Change from full model	Expected effect
Separate encoders	Split backbone per token type	AUC drop
No segment embeddings	Remove $\mathbf{s}_i$	Minor AUC drop
No cross-type attention	Mask across token types	Large AUC drop
Depth scaling ($N = 1, 2, 4, 8$)	Vary backbone depth	Scaling law curve
Width scaling ($d = 32, 64, 128, 256$)	Vary embedding dim	Scaling law curve

3.3 Scaling Law Exploration

I intend the depth and width ablations above to produce a loss-vs-compute curve. If a power-law relationship is observed, I believe this directly supports a submission to the **Scaling Law Innovation Award**, and I would like us to agree on who leads this work.

4 Innovation Award Targets

Unified Block Innovation Award ($45,000) I believe the unified token stream with a single shared backbone constitutes the primary novel claim. The novelty rests on the tokenisation scheme and the absence of type-specific modules.

Scaling Law Innovation Award ($45,000) I propose that we undertake systematic depth and width scaling experiments, combined with fixed compute budget analysis, as our scaling law contribution. These awards are independent of leaderboard rank, which means we can target them regardless of AUC standing.

5 Implementation Plan

I propose to the team the following sequence of steps, subject to team agreement:

1. **Baseline** — We reproduce a standard two-tower (sequence + feature) model on the competition data to establish an AUC baseline we can all verify.
2. **UniToken v0.1** — We implement the unified tokenisation and shared backbone (TensorFlow/Keras, building on `prototype-v0.02`), and share for team review.
3. **Ablations** — we run Table 1 variants together and compare results.
4. **Scaling runs** — we vary N and d; log AUC and FLOPs to build the scaling law curve.
5. **Submission** — leaderboard entry and workshop paper draft, with contributions from all team members.

6 Open Questions —

- Should we discretise continuous features (e.g. dwell time, price) into tokens, or inject them as additive scalings? I do not yet have a strong preference.
- What is the maximum sequence length L the competition data and latency constraints allow? I would like us to establish this before implementation begins.
- Can we agree on TensorFlow/Keras as the implementation framework?
- Who will lead the scaling law experiments, and who will lead the architecture implementation? I would like us to decide this explicitly.